

where `constantExpression` is any constant expression representing a value of type `type`. Thus the declaration

```
var I: Integer = 7;
```

is equivalent to the declaration and statement

```
var I: Integer;
```

```
...
```

```
I := 7;
```

Multiple variable declarations (such as `var X, Y, Z: Real;`) cannot include initializations, nor can declarations of variant and file-type variables.

M.2 The Script Language

In the C-R Function Editor, you can evaluate complex block of statements.

You can use constructions like:

```
If...then...else;
```

```
for I:= ... to .. do ;
```

```
while... do ;
```

```
repeat .... until...;
```

```
break;
```

```
assignment (...:=...;)
```

```
try...finally...end; try...except...end;
```

Each function you create can be a single statement or a block of statements.

When you specify it as a block of statements, your script must conform to the rules of the script language, as follows:

1. Each single statement must end with a semicolon (;)
2. You can use the following statements:

```
variable := expression;
```

```
If logical expression then statement(s) [else statement(s)];
```

for variable := from_expression **to/downto** to_expression **do** statement(s);

while logical_expression **do** statement(s);

repeat statement(s) **until** logical_expression;

try statement(s) **finally** statement(s) **end; try**

statement(s) **except** statement(s) **end;**

inline comments: *// comment... until the end of the line*

nested comments: *{ nested comment }*

Statement(s) in the above declarations states that you can specify either a single statement or a block of statements. The block of statements must be enclosed in **begin ... end** keywords. It is not necessary to enclose the body of the function in **begin .. end**. Cycle statements can use break keyword to break the cycle (break must also end with semicolon.)

M.3 Operands

Expressions may contain the following constant and variable types:

Integer numbers;

Floating point numbers;

Scientific numbers;

Decimal separator for all floating point and scientific-format numbers in expressions, is independent of the Regional Settings of Windows and always is a decimal point ('.').

Boolean values - TRUE or FALSE;

Date type values - values of that type must be put in quotes (' '), and also date separator character is independent of the Regional Settings of Windows and always is a slash - /, i.e. - '01/01/2005'

String values - values of that type must be put in double quotes (" "); If a string contains double quotes, you should double them (i.e., "this is a ""string""");

M.4 Operations

Arithmetical

+, -, ×, /;

div - integer division;

mod - modulo;

^ - power of;

-- negate;

Logical

<, <=, >=, >, <>, =;

and, or, xor, not;

Bitwise

and, or, xor;

~ - negate;

M.5 Arithmetic Functions

ABS(X)	Absolute value
SQR(X)	Square = $X \times 2 = X \times X$
SQRT(X)	Square root
SIGN(X)	Sign of X; =1 for $X > 0$, =0 for $X = 0$, =-1 for $X < 0$
ZERO(X)	=0 for $X = 0$, =1 for $X <> 0$
TRUNC(X)=INT(X)	Integer part
FRAC(X)	fractional part
ROUND(X)	rounds X to the nearest integer value
CEIL(X)	always returns "ceil" integer value
FLOOR(X)	always returns "floor" integer value
DEC(X)	decrements a value X by 1 and returns a new value
INC(X)	increments a value X by 1 and returns a new value
ARG(X,Y)	argument(phase) of X and Y
RADIUS(X,Y)	= $\text{sqrt}(\text{sqr}(X) + \text{sqr}(Y))$
POWER(X,Y)	raises X to a power of Y (Y is a floating point value)
IPOWER(X,Y)	raises X to a power of Y (Y is a integer value)
X^Y	raises X to a power of Y (same as above two functions)
EXP(X)	exponent
LN(X)	natural logarithm
LG(X)	decimal logarithm
LOG(X)	base 2 logarithm
SIN(X)	sine

COS(X)	cosine
TAN(X)	Tangent
COTAN(X)	cotangent
ASIN(X)	Arcsine
ACOS(X)	arccosine
ATAN(X)	arctangent
SINH(X)	hyperbolic sine
COSH(X)	hyperbolic cosine
TANH(X)	hyperbolic tangent

M.6 Aggregate Functions

AVG(X1,X2,...)	returns average value of (unlimited number of) arguments.
MAX(X1,X2,...)	maximum of (unlimited number of) arguments.
MIN(X1,X2,...)	minimum of (unlimited number of) arguments.
SUM(X1,X2,...)	sum of (unlimited number of) arguments.
PROD(X1,X2,...)	product of (unlimited number of) arguments.